iscte TECNOLOGIAS E ARQUITETURA

Software Engineering

GitHubActions for DevOps (Build Automation Tools)

Overview

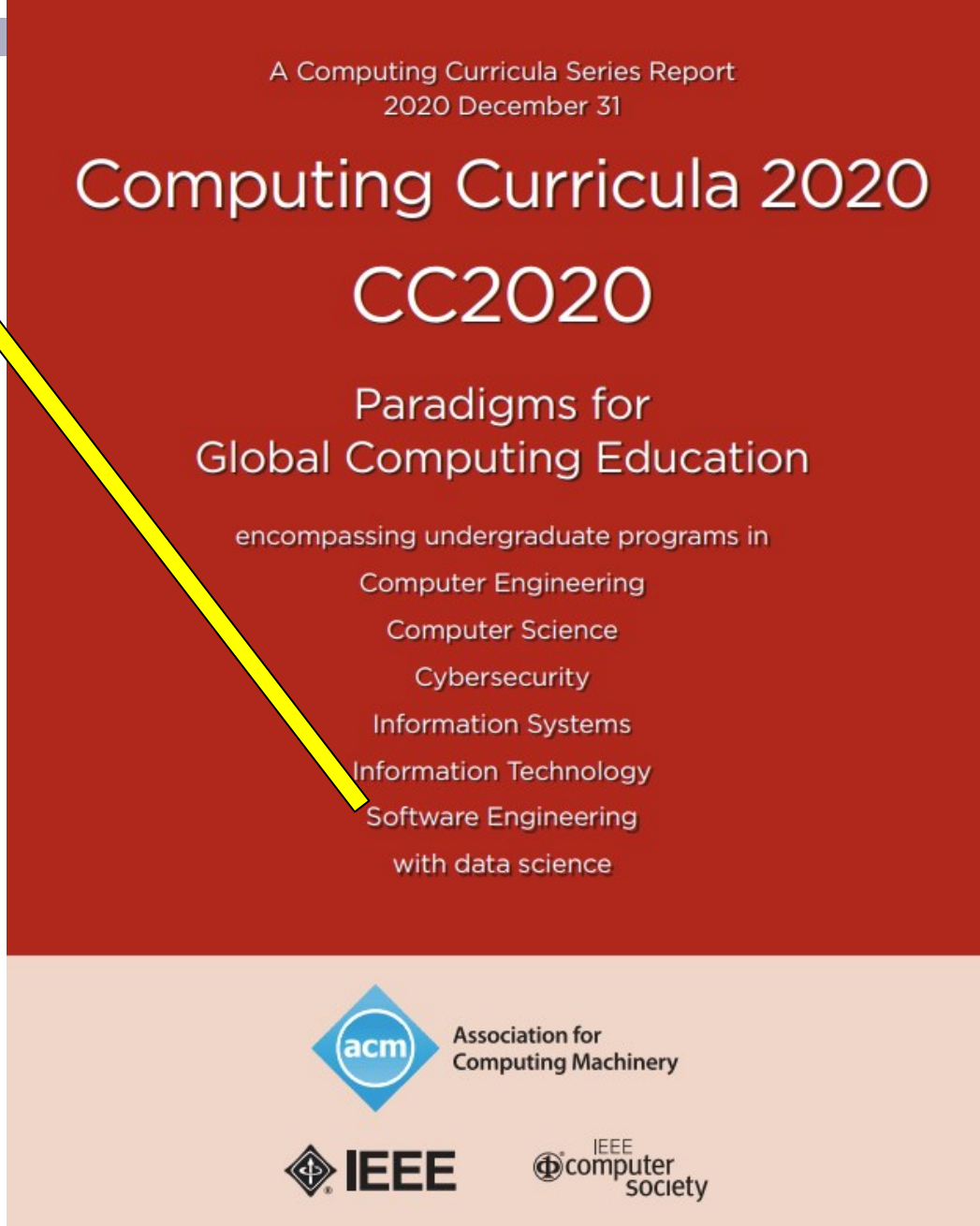
- Why DevOps
- CI/CD pipelines
- Role of automation in DevOps
- What is GitHub Actions
- Using workflows
- YAML
- Events
- Runners
- Exercise

Soft. Eng. Competences

- Software Requirements
- Software Design
- Software Construction
- Software Testing
- Software Sustainment
- Software Process and Life Cycle
- Software Systems Engineering
- Software Quality
- Software Security
- Software Safety
- Software Configuration Manag.
- Software Measurement
- Human-Computer Interaction
- Project Management
- Behavioral Attributes

CDSI

IPM

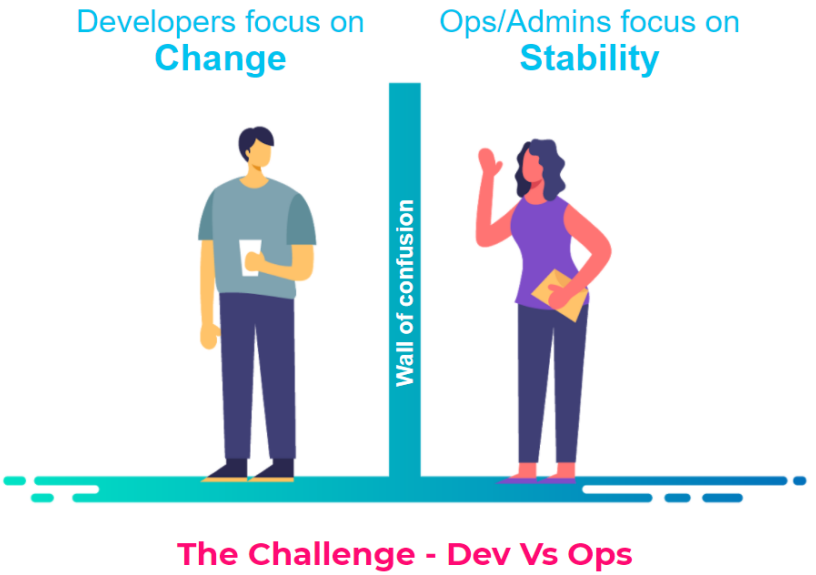


Software Engineering Body of Knowledge

1. Software Requirements
2. Software Architecture
3. Software Design
4. Software Construction
5. Software Testing
6. Software Engineering Operations
7. Software Maintenance
8. Software Configuration Management
9. Software Engineering Management
10. Software Engineering Models and Methods
11. Software Engineering Process
12. Software Quality
13. Software Security
14. Software Engineering Economics
15. Software Engineering Professional Practice
16. Computing Foundations
17. Mathematical Foundations
18. Engineering Foundations

Refers to the detailed creation and maintenance of software through coding, verification, unit testing, integration testing and debugging. [SWEBOK 4.0, Dec. 2022]

Why is DevOps Needed



- Before DevOps, Dev & Op teams worked in isolation
- Dev & Op teams have separate timelines, not in synch, causing delays
- Manual code deployment leads to human errors in production
- Enterprises adopted Agile Dev but Op still looked like waterfall processes
- There is a demand to increase the rate of software delivery
  - Only 17% of teams deliver software fast enough [Forrester Consulting].

Why is DevOps Needed

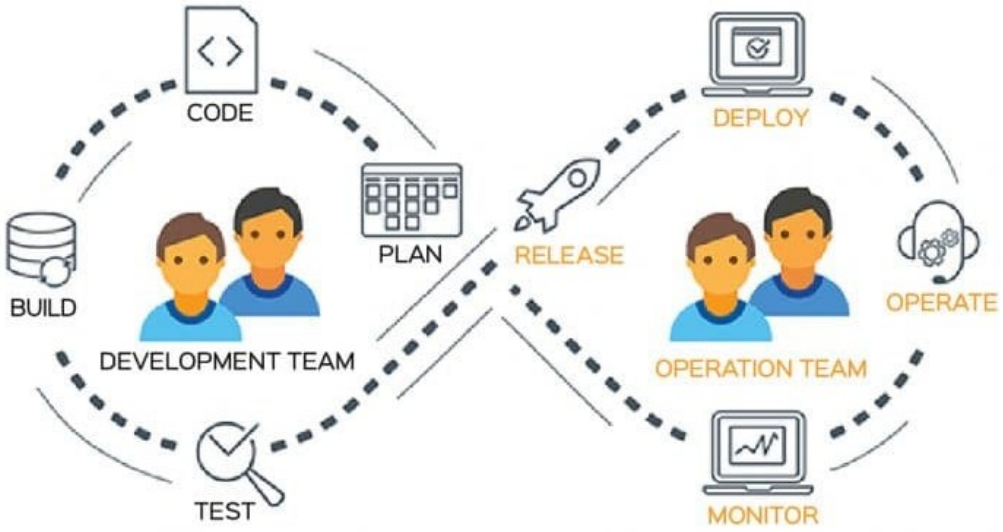
Old Process

- Dev team works on testing, Op team works on deploying the infrastructure.
- Failover, redundancy, storage requirements are skewed, no inputs available from developers who have deep knowledge of the application.
- Op team has no clue on progress of Dev team and develop a monitoring plan as per their understanding.
- Before go-live, the load testing crashes the application. The release is delayed.

DevOps

- Dev & Op teams work together on set-up the new servers.
- Failover, redundancy, disaster recovery, storage requirements are accurate due to inputs from developers.
- Op team is aware of dev progress. Dev & Op teams jointly develop monitoring plan (IT & business needs), including App Performance Monitoring Tools.
- Before go-live, the load testing makes the application a bit slow. Dev team quickly fixes the bottlenecks, and app is released on time.

What is DevOps



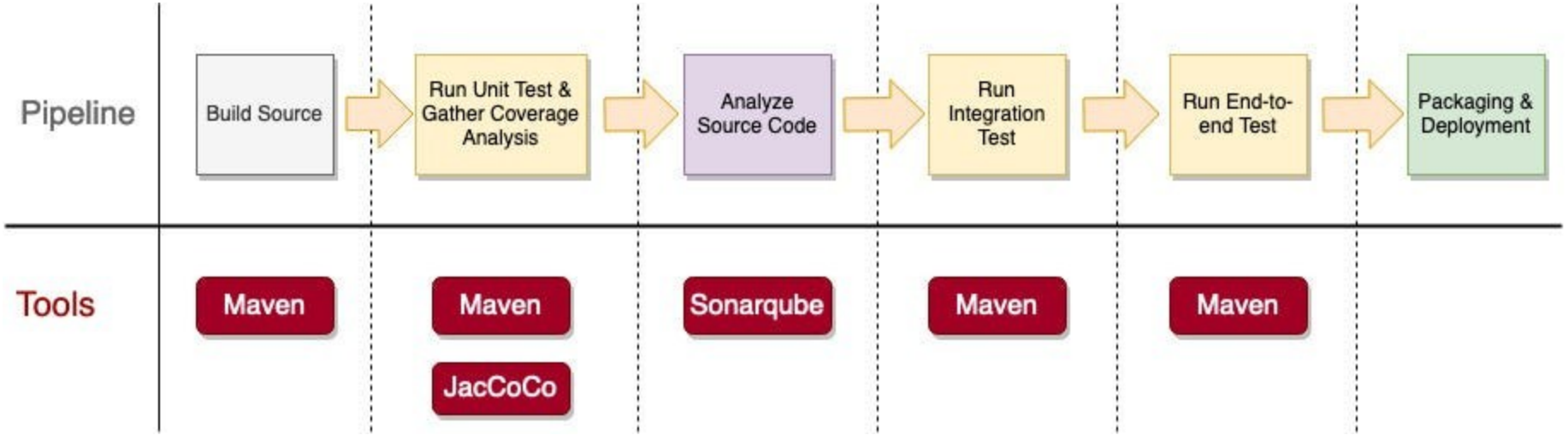
- DevOps is a culture which promotes collaboration between Dev & Op Teams, to deploy code to production faster in an automated & repeatable way.
- Allows Agile Dev Teams to implement Continuous Integration and Continuous Delivery (CI/CD)
- A well-designed DevOps Continuous Delivery pipeline can deploy code from being committed to the code repository to production in the order of minutes, depending on the size of the change.

What is DevOps

Is a field that:

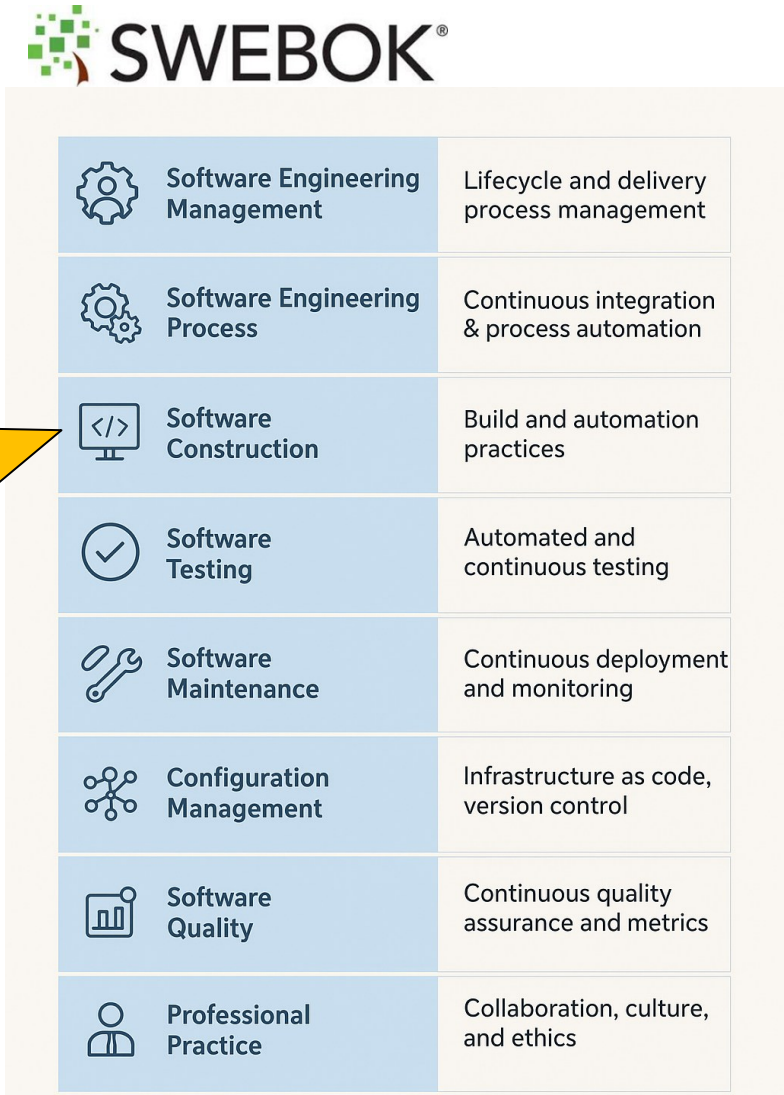
- Integrates development and operation teams (software developers and system administrators, development and production environments)
- Spans several disciplines
- Involves technical & nontechnical cultural aspects (new mindset, tools, new skills)
- Improves collaboration and productivity by automating infrastructure and automating workflows
- Continuously measures application performance (monitoring)
- Has roots in Agile SW development (faster time to market, focus on improving business)

CI/CD pipeline

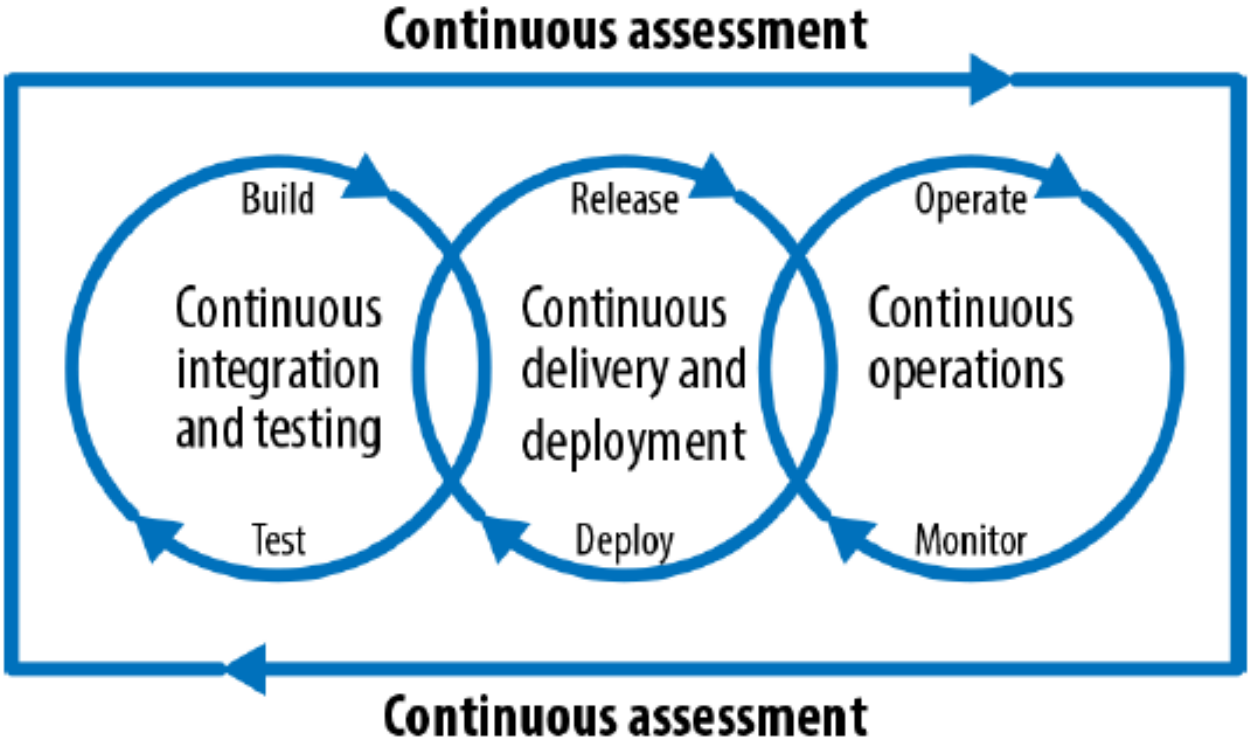


Where DevOps fits in SWEBOK

CI/CD pipelines automate dependency management, build, test, static analysis, code quality gates, packaging, documentation and deployment.
Therefore, DevOps doesn't belong to a single *Knowledge Area (KA)* in the SWEBOK (Software Engineering Body of Knowledge, ISO/IEC TR 19759), but rather cuts across several of them, as shown in the figure.



DevOps



Continuous improvement

What is Github Actions

- Continuous Integration/Continuous Deployment
- Directly integrated with Github
- Much of the work is already done for you (reusability)





GitHub Actions – Workflows explained

- A workflow is an automated process defined by a YAML file in your repository that runs tasks like tests, builds, deployments, etc.
- **Location**: Workflows are stored in the .github/workflows/ directory of your repository.
- **Triggers**: Workflows are triggered by events like:
  - **GitHub Events**: Push, pull request, issue creation, etc.
  - **Manual**: Using the workflow_dispatch event (triggered manually).
  - **Scheduled**: Using the schedule event (e.g., cron jobs).
- **Jobs**: A workflow consists of **one or more jobs**, and each job is made up of a sequence of steps.
  - **Jobs** run on **runners**
  - Jobs can run in parallel or sequentially.

GitHub Actions - Workflows explained

- **Steps**: Individual commands or actions executed within a job. Steps can include running scripts, using pre-built actions, or even calling APIs.
- **Actions**: Reusable units of code that perform specific tasks within a step (e.g., checking out code, setting up environments, etc.).
- **Matrix Builds**: Run the same job across multiple versions of languages or environments, such as testing on different OS versions or Node.js versions.
- **Output & Artifacts**: Workflows can produce output (logs, files) and store them as artifacts for later use.

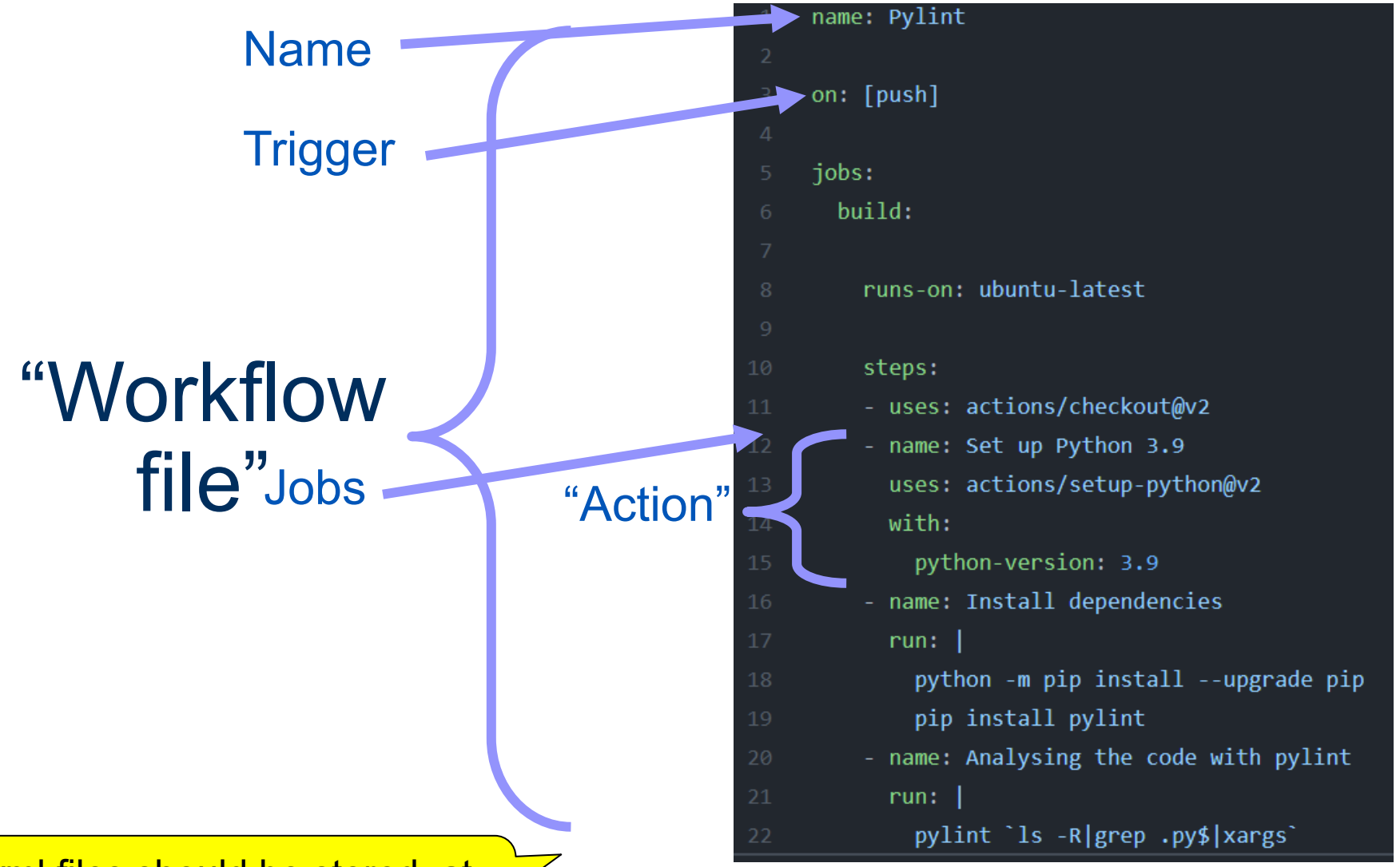
GitHub Actions - Events

- Each event is linked to specific actions or changes in your repository

- **Push** (push): Triggered when code is pushed to a branch or tag.
- **Pull Request** (pull_request): Triggered when a PR is opened, synchronized, or closed.
- **Issue** (issues): Triggered when an issue is opened, edited, or closed.
- **Release** (release): Triggered when a release is published, created, or updated.
- **Schedule** (schedule): Triggered at specified times (cron job).
- **Fork** (fork): Triggered when a repository is forked.
- **Commit Status** (status): Triggered by changes in commit status (e.g., success, failure).
- **Pull Request Review** (pull_request_review): Triggered when a PR review is submitted.
- **Wiki** (wiki): Triggered by changes to the repository wiki.

- More info here

Workflow example in YAML



The .yml files should be stored at {repository-name}/.github/workflows

GitHub Actions - YAML

- **Human-readable**: Simple and clean format for easy reading and writing.
- **Indentation-based Structure**: Uses spaces (typically 2 spaces) to represent nesting and hierarchy.
- **Key-Value Pairs**: Data is represented as key: value.
- **Lists**: Arrays are denoted with - (hyphen).
- **Nested Data**: Supports nested key-value pairs (dictionaries).
- **Comments**: Use # to add comments.
- **Multiline Strings**
- **Literal Style** (|): Retains newlines.
- **Boolean Values**: Represented as true/false, yes/no, on/off.
- **Data Types**: Supports strings, numbers, booleans, arrays, and dictionaries.
- **Variables/Expressions**: Use ${{ variable }} in contexts like GitHub Actions.
- **Flow Style**: Inline representation for lists and mappings (e.g., [a, b, c] or {key: value}).
- **Anchors & Aliases**: Reuse data with & (anchor) and * (alias).
- More info here

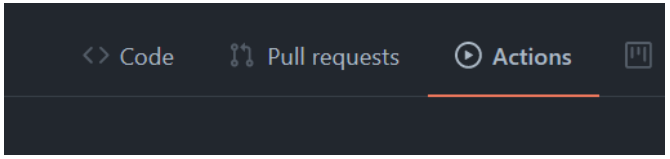
GitHub Actions - Runners

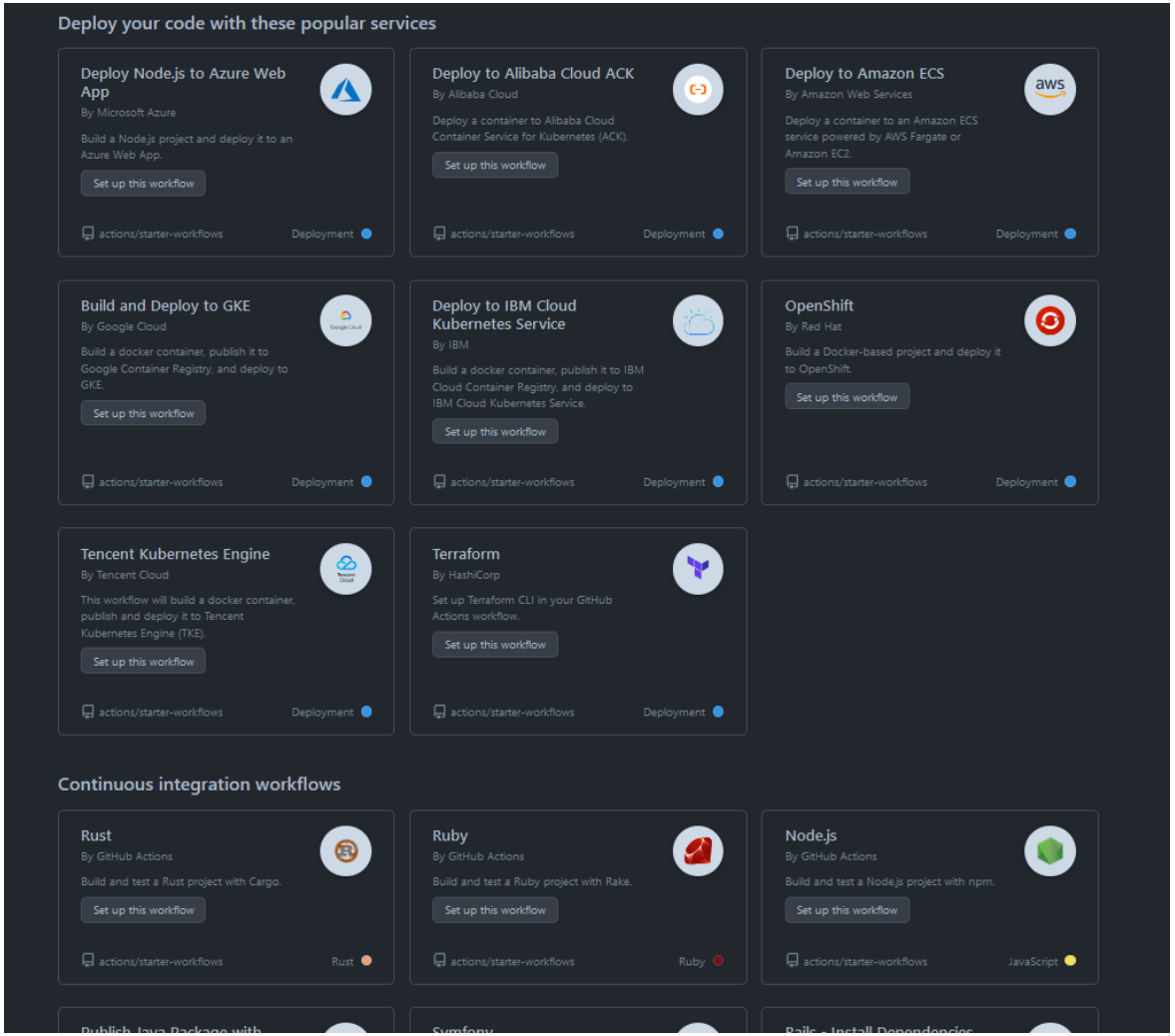
- **Runners** are the machines that run your jobs (scripts, commands, tests, etc.) as part of a GitHub Actions workflow.
- **Types**: There are two types of runners:
  - **GitHub-hosted**: Managed by GitHub, pre-configured with popular software and tools (e.g., Node.js, Python, etc.).
  - **Self-hosted**: Managed by you, can be customized to fit specific needs, and can be located on your own infrastructure.
- **Runs Workflow Jobs**: Each runner can handle multiple jobs in a workflow, and multiple runners can be used for parallel job execution.
- **OS Support**: Runners can be based on various operating systems like Linux, Windows, and macOS.
- **Scaling**: You can add more self-hosted runners for scaling or use GitHub-hosted runners for quick, on-demand usage without setup.

GitHub Actions in Action - Exercise

- Visit the starter workflows
  - https://github.com/actions/starter-workflows

- Follow the quickstart tutorial
  - If your repository doesn't have a **.github/workflows** directory, go to the main page of the repository on GitHub, click Add file, then click Create new file, and name the file **.github/workflows/github-actions-demo.yml**
  - Paste the yaml file example and see the results of the run


Code
Pull requests
Actions


Deploy your code with these popular services
Deploy Node.js to Azure Web App
By Microsoft Azure
Build a Node.js project and deploy it to an Azure Web App.
Set up this workflow
actions/starter-workflows
Deployment
Deploy to Alibaba Cloud ACK
By Alibaba Cloud
Deploy a container to Alibaba Cloud Container Service for Kubernetes (ACK).
Set up this workflow
actions/starter-workflows
Deployment
Deploy to Amazon ECS
By Amazon Web Services
Deploy a container to an Amazon ECS service powered by AWS Fargate or Amazon EC2.
Set up this workflow
actions/starter-workflows
Deployment
Build and Deploy to GKE
By Google Cloud
Build a docker container, publish it to Google Container Registry, and deploy to GKE.
Set up this workflow
actions/starter-workflows
Deployment
Deploy to IBM Cloud Kubernetes Service
By IBM
Build a docker container, publish it to IBM Cloud Container Registry, and deploy to IBM Cloud Kubernetes Service.
Set up this workflow
actions/starter-workflows
Deployment
OpenShift
By Red Hat
Build a Docker-based project and deploy it to OpenShift.
Set up this workflow
actions/starter-workflows
Deployment
Tencent Kubernetes Engine
By Tencent Cloud
This workflow will build a docker container, publish and deploy it to Tencent Kubernetes Engine (TKE).
Set up this workflow
actions/starter-workflows
Deployment
Terraform
By HashiCorp
Set up Terraform CLI in your GitHub Actions workflow.
Set up this workflow
actions/starter-workflows
Deployment
Continuous integration workflows
Rust
By GitHub Actions
Build and test a Rust project with Cargo.
Set up this workflow
actions/starter-workflows
Rust
Ruby
By GitHub Actions
Build and test a Ruby project with Rake.
Set up this workflow
actions/starter-workflows
Ruby
Node.js
By GitHub Actions
Build and test a Node.js project with npm.
Set up this workflow
actions/starter-workflows
JavaScript